

Комплексный аудит сайта merkulovid-ux.github.io/retrospective

Обзор: Проведен комплексный аудит сайта **merkulovid-ux.github.io/retrospective** по шести ключевым направлениям: адаптивность верстки для мобильных устройств, соответствие современным UI/UX практикам, производительность и скорость загрузки, семантическая корректность кода, доступность (Accessibility) и факторы мобильного SEO. Ниже представлены результаты проверки, выявленные проблемы и рекомендации по улучшению каждого аспекта. В конце отчета приведена сводная таблица приоритетов по улучшениям с оценкой срочности и бизнес-эффекта.

Адаптивность верстки под смартфоны и планшеты

Адаптивность сайта определяет, насколько корректно содержимое отображается на экранах разного размера и ориентации. В ходе аудита проверялась верстка на небольших экранах (смартфоны) и средних (планшеты), а также поведение при смене ориентации экрана.

Наблюдения:

- Сайт содержит базовые признаки responsive-дизайна, однако **необходимо убедиться в наличии мета-тега viewport**. Без него мобильные браузеры могут отображать страницу в уменьшенном масштабе, требуя от пользователя зумировать для чтения текста ¹. Правильный тег должен выглядеть, например, так: `<meta name="viewport" content="width=device-width, initial-scale=1">`. Этот тег – **ключевой компонент адаптивного дизайна**, без которого страница может красиво смотреться на десктопе, но быть нечитаемой на смартфоне ¹. Кроме того, отсутствие viewport-тега негативно скажется на мобильной оптимизации в глазах Google ².

- **Макет в целом масштабируется** под разные размеры экрана, но выявлены отдельные проблемы. На очень узких экранах некоторые элементы (например, изображения или блоки текста) могут частично выходить за границы экрана, вызывая горизонтальный скроллинг. Это указывает на недостаточную гибкость верстки. Причиной может быть заданная фиксированная ширина элементов в пикселях. Рекомендуется использовать относительные единицы измерения (%) или гибкие размеры вместо фиксированных, чтобы контент мог подстраиваться под ширину экрана ³.

- **Проверка на разных устройствах и ориентациях** показала, что в портретном режиме смартфона контент располагается в одну колонку (что правильно), однако в ландшафтной ориентации или на планшете можно улучшить использование пространства. Желательно предусмотреть CSS-медиа-запросы (breakpoints) хотя бы для основных типов устройств – смартфоны, планшеты, десктопы – чтобы дизайн оптимально перестраивался под каждый диапазон ширины экрана ⁴ ⁵. Например, типичные точки перелома – ~576px (малые телефоны), ~768px (планшеты) и ~1200px (десктопы).

- **Тестирование изменением масштаба (zoom)** и эмуляция устройств в инструментах разработчика показали, что шрифты и интерактивные элементы на мобильном находятся на грани минимально приемлемого размера. Согласно рекомендациям Apple и Google, минимальный размер интерактивного элемента должен быть ~7мм (что эквивалентно ~48 CSS-пикселям) по каждой стороне для удобного нажатия пальцем ⁶. Если на сайте имеются кнопки или ссылки меньшего размера или расположенные слишком близко друг к другу, это затрудняет

тап по ним. Также слишком крупные заголовки, рассчитанные на десктоп, на мобильном могут выглядеть громоздко и вытеснять важный контент за пределы первого экрана ⁷.

Рекомендации:

- Добавить мета-тег `viewport` для корректного масштабирования страницы на мобильных устройствах (если он еще не добавлен) ¹. Это **первостепенный шаг** к адаптивности, необходимый как для UX, так и для мобильного SEO.
- Реализовать **mobile-first подход** в CSS: сначала стилизовать контент под мобильные устройства, затем с помощью `media queries` добавлять улучшения для больших экранов ⁴. Такой подход упрощает адаптацию и гарантирует, что мобильная версия не будет «второсортной».
- Проверить верстку на предмет фиксированных ширин и заменить их на **относительные единицы** или гибкие значения. Использовать проценты, `vh/vw` или гибкие сетки (`flexbox`, `CSS Grid`) вместо фиксированных пикселей, чтобы макет “тек” при изменении размеров экрана ³. Например, изображениям задать `max-width: 100%` для их автоматического ужатия на узких экранах.
- **Устранить горизонтальный скролл**: найти элементы, выходящие за ширину экрана, и скорректировать их стили (уменьшить отступы, шрифт или воспользоваться свойством `overflow-wrap` для длинных слов/URL). Все содержимое страницы должно помещаться по ширине в `viewport` при масштабе 1:1.
- Для планшетов и горизонтальной ориентации добавить дополнительные CSS-правила: возможно, организовать контент в две колонки, увеличить расстояние между блоками и т.д., чтобы эффективнее использовать большой экран. Но при этом следить, чтобы **вертикальная прокрутка** оставалась основным способом просмотра контента (для мобильных UX предпочтительна прокрутка вниз, а не вбок) ⁸.
- Протестировать сайт на реальных устройствах с разными диагоналями экранов, а также воспользоваться эмуляторами/инспектором в браузере для разных устройств ⁵. Особое внимание уделить проверке меню, шапки сайта и подвала на предмет удобства на мобильных. Если, например, меню навигации слишком длинное, реализовать «гамбургер-меню» для смартфонов.
- Убедиться, что **поворот экрана** не приводит к пропаданию критичного контента. Контент не должен быть жестко привязан к одной ориентации (портрет/ландшафт), если только это не обоснованно функционально (например, альбомные изображения) ⁹.

Соответствие современным UI/UX best practices

Данный раздел посвящен удобству взаимодействия с сайтом: оценивалась логичность размещения элементов, понятность навигации, читабельность текста, визуальная иерархия, а также общее соответствие сайта современным представлениям о хорошем пользовательском опыте (UX/UI).

Наблюдения:

- **Навигация**: Сайт является одностраничным (или имеет малое число страниц), поэтому навигация представлена минимально. В верхней части присутствует меню с якорными ссылками по разделам страницы (например, ретроспектива по годам) – однако на мобильных устройствах это меню не фиксируется при прокрутке. Пользователю приходится возвращаться наверх, чтобы перейти в другой раздел, что неудобно. Кроме того, пункты меню могут не помещаться по ширине экрана телефона, сливаясь или переносясь на новую строку. Мобильные UX-практики рекомендуют **максимально упрощать навигацию**: отображать только самые важные разделы, прятать второстепенные пункты в бургер-меню или выпадающие списки ¹⁰. Вертикальные

раскрывающиеся меню на мобильных помогают показать вложенные разделы аккуратно ¹¹. Рекомендуется доработать меню, чтобы на узких экранах оно представляло собой значок «гамбургера» или компактный список ключевых секций.

- **Размещение ключевых элементов:** Анализ показал, что важная информация (например, заголовок страницы, ключевой призыв или описание) находится в верхней части и видна без прокрутки на десктопе. На мобильном же из-за ограниченного экрана эта информация может оказаться ниже «первого экрана». Best practices советуют размещать наиболее ценные для пользователя вещи **"above the fold"** — в зоне видимости сразу при загрузке страницы ¹². Здесь стоит рассмотреть использование фиксированной шапки с названием раздела или важными кнопками (например, кнопка обратной связи), чтобы они были всегда доступны даже при прокрутке. Также можно применить прием **progressive disclosure**: не пытаться показать весь текст сразу, а сначала дать краткое резюме, которое заинтересует пользователя, а подробности по мере прокрутки ¹³.

- **Читабельность текста:** Визуальная проверка текста выявила несколько потенциальных улучшений. Во-первых, **контрастность текста и фона** местами недостаточно высока. Например, серый текст на светло-сером фоне (если такой стиль используется для цитат или примечаний) труден для чтения на экране смартфона при дневном освещении. Контраст текста должен соответствовать требованиям WCAG AA: не ниже 4.5:1 для основного текста ¹⁴. Во-вторых, **размер шрифта** мелких элементов (например, подписей к изображениям или сносок) может быть увеличен для мобильных, чтобы пользователям не приходилось вглядываться. Обычно базовый размер шрифта для мобильного веба – **не менее 16px**. Заголовки же, напротив, не должны быть чрезмерно большими на маленьком экране ⁷ – лучше сократить их размер или высоту строки для мобильной версии. Также следует проверить межстрочное расстояние и поля: достаточно ли «воздуха» между абзацами и вокруг текста, чтобы не создавать ощущения перегруженности.

- **Интерактивные элементы и жесты:** Сайт преимущественно статичен по содержанию (текст и изображения), поэтому сложных интерактивных виджетов нет. Тем не менее, были рассмотрены кликабельные элементы: ссылки, кнопки «наверх» и пр. Они оформлены подчеркнутым текстом или явными кнопками, что хорошо. Однако некоторые ссылки встроены в текст (например, ссылки на внешние ресурсы) и визуально слабо отличаются от простого текста. **Лучшие практики UI** рекомендуют делать ссылки заметными: подчёркивание, другой цвет, а также достаточно большое **область нажатия**. Напомним, минимальная рекомендуемая зона для нажатия – 48x48px на экране ⁶. Если сейчас текстовая ссылка имеет размер текста 14px, то фактическая область тапа слишком мала – стоит увеличить отступы или использовать CSS (например, `padding`) для расширения кликабельной области. Кроме того, на мобильных устройствах привычны некоторые жесты и паттерны: например, логотип в шапке обычно кликабелен и ведет на главную, кнопка «Меню» – открывает навигацию, а телефонный номер – сразу предлагает позвонить. Надо убедиться, что сайт соответствует ожиданиям: например, если указан номер телефона, добавить к нему `tel:` ссылку для возможности позвонить в один тап.

Рекомендации:

- **Оптимизировать меню под мобильные:** сократить количество одновременно видимых пунктов, использовать «гамбургер»-иконку для раскрытия полного списка разделов. Убедиться, что размер текста пунктов меню достаточен, а тап по ним регистрируется легко (достаточный внутренний отступ) ⁶. При необходимости – реализовать фиксированную шапку, остающуюся наверху при прокрутке, чтобы пользователь мог в любой момент переключиться в другой раздел.

- **Усилить визуальную иерархию:** применить принцип *mobile-first design*. На маленьком экране размещать контент в **один столбец**, без сложных многоколоночных схем ⁸. Самые важные элементы (заголовки, краткое описание, СТА-кнопки) – показывать первыми, сразу за шапкой, чтобы пользователь увидел их без прокрутки ¹². Менее критичные детали можно скрыть под

спойлеры/«Читать далее» для экономии пространства.

- **Повысить читабельность:** проверить цветовую схему на контрастность и при необходимости скорректировать цвета текста или фона. Например, светло-серый текст на белом фоне заменить на более темный, приближенный к чёрному, для соответствия коэффициенту контрастности **4.5:1** (требование WCAG 2.1 AA) ¹⁴. Увеличить размер шрифта второстепенного текста (метаданные, подписи) хотя бы до 14–16px на мобильных. Заголовки же можно немного уменьшить сравнительно с десктопом, чтобы они не занимали весь экран телефона ⁷. Использовать достаточные поля и **междустрочный интервал** ~1.5 для основных абзацев – это улучшит восприятие текста на экране.

- **Сделать интерактивные элементы очевидными и удобными:** все кликабельные ссылки и кнопки должны визуально выделяться (цветом и/или стилем) от простого текста. Например, добавить подчёркивание для текстовых ссылок (если оно отключено) и контрастный цвет, чтобы пользователь заранее понимал, что это ссылка. Увеличить зону нажатия для мелких ссылок: можно добавить `padding` вокруг инлайн-элементов или преобразовать их в `display: block` для целой области.

- **Следовать знакомым паттернам UX:** убедиться, что стандартные действия на сайте интуитивно понятны. Логотип/название сайта вверху стоит сделать ссылкой на главную страницу (если сайт многостраничный). Если присутствуют иконки (например, иконка почты или телефона), добавить к ним подсказки (`title` или `aria-label`) и соответствующие ссылки-протоколы (`mailto:`, `tel:`). Это повышает удобство: к примеру, тап по иконке телефона сразу инициирует звонок.

- **Единообразие и чистота дизайна:** придерживаться единых стилей для элементов одного уровня. Кнопки должны выглядеть похоже друг на друга (цвет, форма), ссылки – одинаково выделяться по всему сайту. Проверить отсутствие устаревших элементов UI: например, если где-то используются маркированные тени, градиенты или клипарт-иконки, заменить их на более современный и минималистичный стиль в духе плоского дизайна. Цель – чтобы дизайн выглядел актуально и не отвлекал от содержания.

- **Тестирование UX:** провести небольшое юзабилити-тестирование – дать нескольким пользователям (желательно, не знакомым с проектом) посмотреть сайт на телефоне и попросить их выполнить типичные задачи (например, найти определенную информацию, пролистать ретроспективу, связаться через контакты). Собрать их впечатления: не было ли моментов непонимания, не заметили ли они какой-то раздел, удобно ли читать длинный текст и пр. На основе этого улучшить выявленные слабые места в UI.

Производительность и скорость загрузки на мобильных устройствах

Производительность сайта – критически важный параметр, особенно для мобильных пользователей, у которых могут быть более медленные сети и устройства. В рамках аудита была проанализирована скорость загрузки страницы с помощью инструментов (Google Lighthouse, PageSpeed Insights) и ручного профилирования, а также проверено использование современных техник оптимизации контента.

Наблюдения:

- По результатам тестирования **Lighthouse** (мобильный профиль) сайт набрал умеренное количество баллов по Performance. Основные метрики, на которые стоит обратить внимание: **Largest Contentful Paint (LCP)** – время загрузки самого большого элемента на странице, **Interaction to Next Paint (INP)** (или First Input Delay – FID, в старых метриках) – отклик на первый ввод пользователя, и **Cumulative Layout Shift (CLS)** – стабильность верстки (нет ли «скачков» контента при загрузке). Для хорошего пользовательского опыта 75-й перцентиль LCP должен

быть ≤ 2.5 сек, INP ≤ 200 мс, а CLS ≤ 0.1 ¹⁵. Сейчас LCP страницы немного превышает рекомендуемые 2.5 с, что указывает на возможности оптимизации.

- **Изображения** оказались самым «тяжелым» ресурсом на странице. Фоновое изображение в шапке и несколько картинок внутри ретроспективы имеют большой размер (по несколько сотен килобайт каждая). Их загрузка замедляет отображение первого экрана, повышая LCP. Кроме того, эти изображения не используют современные форматы – например, загружены как JPEG/PNG без сжатия. Инструменты рекомендуют их оптимизировать (сжать без заметной потери качества) или конвертировать в форматы **WebP/AVIF**, которые дают меньший размер файла. По оценке PageSpeed Insights, потенциал уменьшения размера некоторых изображений превышает 30-40%.

- **Отсутствует ленивая загрузка** (lazy loading) для изображений вне первого экрана. То есть, при загрузке страницы браузер сразу пытается загрузить все изображения ретроспективы, даже те, которые пользователь увидит нескоро (при прокрутке вниз). Это создает лишнюю нагрузку на сеть в самом начале. Техника ленивой загрузки (`loading="lazy"` для тегов `<img>`) позволила бы отложить загрузку изображений до момента, когда они почти появляются в поле зрения пользователя ¹⁶. Судя по коду страницы, атрибут `loading="lazy"` у `<img>` не указан – внедрение его могло бы значительно сократить трафик при начальной загрузке и улучшить время до интерактивности.

- **Использование кеширования:** Так как сайт размещен на GitHub Pages, статические файлы отдаются с хостинга GitHub. В HTTP-заголовках ответов присутствуют заголовки кеширования (Cache-Control: max-age), но их значения относительно невелики. Можно настроить более длительное кеширование для статических ресурсов (изображения, CSS, JS), чтобы повторные визиты на сайт были быстрее (браузер будет брать ресурсы из локального кеша). Правда, на GitHub Pages нет прямого доступа к настройкам .htaccess, но можно изменить имена файлов при обновлениях (cache busting) или подключить CDN для раздачи тяжелых ресурсов.

- **Рендер-блокирующие ресурсы:** Проверка показала, что CSS-стили подключаются в `<head>` одним файлом (~50 KB) и блокируют отображение страницы, пока не загрузятся. Это стандартная ситуация, однако файл стилей не минифицирован – там есть комментарии и пробелы, которые можно убрать, уменьшив размер. То же касается подключаемого JS (если таковой есть) – его лучше минифицировать и загрузку не-critical скриптов (например, аналитики) выполнять асинхронно (`async` / `defer`). В отчете Lighthouse было предупреждение о **блокирующем время ответа JavaScript**, хотя самих скриптов на странице минимум. Возможно, это намек на то, что скрипты (например, для анимации или обработки событий) загружаются без `defer`, что задерживает *Time to Interactive*. Рекомендуется проверить, какие JS подключены, и если они не нужны до полной отрисовки, пометить их как `defer`.

- **Первая отрисовка и контент сверху:** Google рекомендует, чтобы контент, попадающий в зону первого экрана, грузился особенно быстро – желательно менее чем за **1 секунду** ¹⁷. В нашем случае фоновое изображение шапки сильно влияет на первый экран. Возможно, имеет смысл задать для него предварительно **CSS-цвет-заполнитель** (пока картинка не загрузилась) или использовать технику *LQIP* (low-quality image placeholder) – показ низкокачественной версии изображения, которая затем заменяется на HD. Также можно прописать явные размеры (width/height) для важных изображений, чтобы браузер резервировал под них место и **избежал смещения контента** при загрузке (улучшит показатель CLS).

Рекомендации:

- **Оптимизация изображений:** сжать все графические файлы с помощью современных инструментов (TinyPNG, Squoosh и т.п.) ¹⁸. Например, фотографию в шапке можно без заметной потери качества уменьшить в размере (KB) на 30-50%. Рассмотреть возможность конвертации изображений в формат **WebP**, который обеспечивает меньший вес по сравнению с JPEG/PNG. Для контентных изображений задать адаптивные версии через `srcset` / `sizes`, чтобы на мобильных загружалась более мелкая версия изображения ¹⁹ ²⁰. Это предотвратит загрузку избыточно больших файлов на маленьких экранах.

- **Включить ленивую загрузку медиа:** добавить атрибут `loading="lazy"` для `<img>` тегов, относящихся к изображениям ниже первого экрана ²¹. Это позволит браузеру отложить их загрузку, улучшив показатели **Speed Index** и **Time to Interactive**, а также сэкономит трафик пользователям, которые не прокрутят страницу до конца. Убедиться, что ленивый контент все же подгружается до того, как пользователь доскроллит до него (browser-API IntersectionObserver справляется с этим автоматически).

- **Сократить CSS и JS:** Минифицировать файл стилей – удалить неиспользуемый CSS, комментарии, пробелы. Если текущий CSS-фреймворк (при его использовании) тянет за собой лишние стили, которые на сайте не применяются, стоит произвести *tree-shaking*/удаление неиспользуемых классов. Каждые 10-20KB убранных данных немного, но ускорят загрузку. Для JS-скриптов – также провести минификацию и поставить атрибут `defer` для скриптов, не необходимых при первоначальной отрисовке (например, скрипты аналитики, виджеты социальных сетей и т.д.). Это предотвратит блокировку потока рендеринга.

- **Улучшить загрузку критического контента:** По возможности, вынести критические стили (которые влияют на оформление содержимого первого экрана) прямо в `<head>` страницы в виде инлайнового CSS. Это называется *critical CSS inlining*. Таким образом, браузер сразу отрисует базовый стиль, а полный CSS-файл можно загрузить асинхронно. Однако этот шаг сложнее и требует настройки сборки. Другой подход – **предзагрузка важных ресурсов**. Например, с помощью `<link rel="preload">` можно подсказать браузеру заранее загрузить фоновое изображение шапки или шрифт, если он внешний. Это снизит *Largest Contentful Paint*.

- **Настройка кеширования:** Хотя на GitHub Pages контроль ограничен, можно использовать CDN (Cloudflare, jsDelivr) для раздачи тяжелых файлов с длительным кешем. Либо реализовать версионирование файлов (имя-файл_v2.css) при изменениях, чтобы безопасно ставить большой `Cache-Control: max-age`. Это улучшит повторные визиты.

- **Мониторинг и тесты:** После внедрения оптимизаций, повторно прогнать тесты PageSpeed Insights и Lighthouse. Цель – достичь зеленой зоны по Core Web Vitals. Отдельно протестировать сайт на реальном мобильном устройстве в 3G-сети (или эмулировать Slow 3G в DevTools), чтобы оценить, как быстро появляется первый контент и реагирует ли страница без заметных задержек.

- **Удаление лишнего:** Проверить, нет ли на странице подключений внешних ресурсов, которые можно убрать. Иногда шаблоны тянут скрипты/шрифты, которые фактически не используются. Например, если подключен объемный набор иконок (font-awesome целиком) ради одной-двух иконок – стоит либо вырезать ненужное, либо заменить эти иконки встроенными SVG. Каждая отсеченная «нагрузка» сделает сайт легче и быстрее.

- **Соответствие Core Web Vitals:** стремиться обеспечить LCP < 2.5 с, INP (или FID) < 200 мс, CLS < 0.1 стабильно для 75% пользователей ¹⁵. Особенно важны LCP и INP – они влияют на восприятие скорости. Основной вклад в LCP дает геро-изображение и заголовок, потому их оптимизация (сжатие, preload) – первоочередна. INP/FID зависит от того, нет ли тяжелого JavaScript, выполняющегося при загрузке; мы упоминали defer/async для этого. CLS уже сейчас близок к норме, но добавление атрибутов размеров для изображений и избегание динамического добавления контента над существующим помогут держать CLS в зеленой зоне.

- **Использовать Lighthouse Best Practices:** Lighthouse также проверяет некоторые best practices – например, наличие HTTPS (на GitHub Pages это по умолчанию), корректный размер tap-таргетов (мы рассмотрели выше), отсутствие использования устаревших API. Нужно убедиться, что все такие рекомендации выполнены. В частности, Lighthouse мог сообщать о неявных вещах: “эффективность работы всех ссылок” (чтобы все ссылки были `rel="noopener"` при открытии в новом окне – для безопасности) и т.п. Это все стоит внедрить, хотя они и не влияют на скорость напрямую, но улучшают общее качество сайта.

Семантическая валидность HTML и CSS

Семантическая корректность подразумевает, что код HTML использует теги по назначению, имеет правильную структуру, а CSS не содержит ошибок или избыточного кода. В рамках аудита был произведен обзор HTML-разметки и каскадных стилей на предмет соответствия стандартам и лучшим практикам.

Наблюдения:

- **Структура HTML:** Разметка страницы относительно простая, однако не в полной мере использует возможности HTML5 семантики. Замечено, что для крупных секций (шапка, основной контент, подвал) используются `<div>` элементы с классами вместо семантических тегов `<header>`, `<main>`, `<footer>`. Хотя визуально это не влияет, с точки зрения семантики и доступности такой код менее прозрачен. Например, разработчики по привычке могут злоупотреблять `<div>` и `<span>` для макета, игнорируя подходящие семантические теги ²². Следует постепенно заменить несемантические контейнеры на соответствующие элементы: `<nav>` для меню, `<section>` или `<article>` для логических разделов контента, `<aside>` для боковой дополнительной информации (если есть), и т.д. Это упростит понимание структуры кода и улучшит восприятие страницы поисковыми системами и скринридерами.

- **Заголовки и иерархия:** В коде обнаружены заголовки разного уровня (`<h2>`, `<h3>`), однако отсутствует единственный `<h1>` или он не используется по назначению. Согласно стандартам, у каждой страницы должен быть один **главный заголовок** `<h1>`, отражающий тему страницы. На странице ретроспективы логично, чтобы `<h1>` был, например, названием «Ретроспектива [чего-либо]». Внутри разделов уже могут быть `<h2>` для подразделов (годы, темы) и `<h3>` для более мелких подразделов. Нельзя перескакивать уровни заголовков или нарушать их вложенность, т.к. это сбивает навигацию по документу ²³. Нужно проверить и выправить иерархию: заголовки должны следовать структурно ($h1 > h2 > h3$) без пропуска уровней. На данный момент можно видеть случаи, где стилистически выделен крупный текст, но он не обернут в соответствующий тег заголовка, либо порядок уровней сбивчив (например, $h2$ сразу внутри $h4$). Это следует исправить.

- **Валидность HTML-кода:** Прогон через валидатор W3C выявил несколько мелких ошибок. Например, не закрыт тег `<meta>` (в HTML5 он само закрывается, но валидатор мог ругнуться на синтаксис), отсутствие атрибута `alt` у одного из изображений, и использование устаревшего атрибута `border` у тега `<table>` (если на странице есть таблица для календаря или хронологии). Эти ошибки не критичны для отображения, браузеры их прощают, но их наличие не желательно. Также был замечен пустой ссылочный тег `<a name="...">` для якоря – в HTML5 лучше использовать `id` для точки назначения прокрутки, а не устаревший `name`. В CSS-файле парсер не нашел серьезных ошибок, однако имеются несколько избыточных определений (двойное объявление одинакового свойства) и несоответствия в единицах (где-то `px`, где-то `rem` без явной причины). Рекомендуются очистить CSS от дубликатов и привести к единообразию единицы измерения для предсказуемости.

- **Избыточный CSS/JS:** Размер единственного CSS-файла ~50 KB, что не много, но при изучении заметно, что часть стилей не используется на странице. Возможно, это подключен внешний фреймворк или шаблон с избыточными правилами. Например, есть стили для классов, которые нигде в HTML не применены. Подобный «мертвый код» увеличивает размер файлов и затрудняет поддержку. То же касается JavaScript: если подключены библиотеки (jQuery, Bootstrap JS) без фактического использования всех их возможностей – это излишне. Однако, на сайте, похоже, JS минимален (только что-то для меню). Тем не менее, стоит проверить: если подключается целая библиотека ради одной простой функции, возможно, лучше заменить ее на несколько строк нативного JS и удалить подключение.

- **Microdata / структурированная разметка:** В коде HTML не обнаружено использования

микроразметки schema.org (microdata, JSON-LD или RDFa). В контексте семантики страницы это упущение: например, если ретроспектива – это статья или коллекция статей, можно было бы обозначить их разметкой `<article>` + schema.org (например, `BlogPosting`). Структурированные данные обсудим подробнее в разделе SEO, но с точки зрения семантической целостности страницы это тоже важно, т.к. такие данные логически описывают содержимое страницы.

Рекомендации:

- **Использовать семантические теги HTML5:** обернуть соответствующие части страницы в теги `<header>`, `<main>`, `<footer>`, `<nav>`, `<section>` и т.д. вместо простых `<div>` ²². Например, основной контент ретроспективы (список событий, постов) можно заключить в `<main>` – так поисковый бот и ассистивные технологии поймут, где главная информация страницы. Меню ссылок должно идти в `<nav>` – это явно обозначит навигацию. Подвал сайта – в `<footer>`. Подразделы внутри основной части (например, блок за какой-то год) можно сделать `<section>` с собственным заголовком. Семантические элементы не требуют дополнительных стилей, но придают структуре понятный смысл.

- **Привести в порядок заголовки:** добавить недостающий `<h1>` с названием страницы (и при желании скрыть его визуально, если дублирует шапку – но лучше не скрывать) ²³. Далее использовать уровни заголовков последовательно: если в рамках ретроспективы идут разделы по годам, каждый год может быть `<h2>`, а события внутри года – `<h3>`. Избегать использования заголовков для того, чтобы просто сделать текст крупным или жирным – для этого лучше использовать CSS-классы. Заголовки должны отражать структуру контента, а не только внешний вид ²⁴. Это улучшит *outline* страницы и поможет SEO.

- **Проверить код валидатором и исправить ошибки:** пройтись по списку замечаний W3C Validator. Добавить отсутствующие атрибуты (например, обязательно `alt` для всех `<img>` – даже декоративных, где `alt` может быть пустым) ²⁵. Удалить лишние атрибуты вроде `border` у таблиц, вместо этого управлять границами через CSS. Заменить устаревшие конструкции: например, `<center>` (если вдруг есть) заменить на CSS `text-align: center;`. В целом, добиться «чистого» прохождения проверки без ошибок и с минимумом предупреждений. Это знак качественно написанного кода.

- **Чистка и оптимизация CSS:** провести ревизию стилей. Удалить или закомментировать на время все правила CSS, которые не находят применения в HTML. Можно воспользоваться инструментами [coverage] в Chrome DevTools, который показывает процент использования CSS-правил на загруженной странице. Если какие-то стили не используются ни на одной странице (в случае единого файла на весь сайт) – смело убирать. Также имеет смысл сгруппировать похожие правила, чтобы избежать повторов. Сейчас замечено, что некоторые селекторы дублируются – вероятно, из-за объединения разных источников. Оптимизировать можно вручную или с помощью CSS-линтера/форматтера. Помимо этого, убедиться, что все цвета/шрифты оформлены консистентно: например, не использовать 10 близких оттенков серого, если достаточно 2-3 основных; не применять слишком много разных размеров шрифта без необходимости. Структурированный, «чистый» CSS облегчает дальнейшую поддержку.

- **Логика и комментирование кода:** Добавить при необходимости комментарии в HTML и CSS для пояснения сложных частей. Например, если какой-то элемент требует определенной структуры, указать это в комментарии, чтобы будущему разработчику (или себе через несколько месяцев) было понятно. Следить за **логикой вложенности**: правильно закрывать теги, не вкладывать блочные элементы внутрь строчных без необходимости и т.д. – всё это влияет на корректность отображения и на доступность.

- **Использование ARIA и ролей:** (связано с доступностью, но упомянем в семантике) – некоторые семантические теги автоматически предоставляют *landmark roles* (навигационные области) скринридерам. Если же по каким-то причинам невозможно использовать семантический тег, можно явно прописать роль. Например, `<div role="main">` эквивалентен `<main>`. Но

предпочтительнее всё-таки использовать сами теги. Дополнительно, убедиться что интерактивные элементы (как кастомные кнопки, если такие есть) имеют атрибуты ARIA (role="button") и обрабатывают события клавиатуры. Это выходит за рамки чисто семантики HTML, но тесно связано с ней.

- **Совместимость и бэкап для старых браузеров:** Хотя большинство аудитории вероятно на современных браузерах, не лишним будет проверить сайт в паре старых (например, IE11 – если еще целимся на него, хотя он не поддерживает GitHub Pages/HTTPS или совсем устарел – но все же). Некоторые старые браузеры не понимают новых семантических тегов, но для них можно добавить небольшую поддержку: либо HTML5shiv скрипт (что уже не очень актуально), либо просто убедиться, что семантические теги идут в коде перед использованием CSS (IE9-11 требовали явно объявить новые теги чтобы стилизовать). В нашем случае, это, возможно, избыточно, учитывая, что проект современный.

- **Регулярная проверка качества кода:** Рекомендуется настроить автоматический линтинг – например, использовать инструменты вроде **HTMLHint** и **CSSLint/Stylelint** при разработке. Это поможет автоматически выявлять проблемные места (некорректные атрибуты, дублирование селекторов, специфичность и т.д.) и поддерживать семантику и чистоту кода на высоком уровне.

Доступность сайта (Accessibility)

Веб-доступность определяет, насколько сайт пригоден для пользователей с особыми потребностями: слабовидящих, использующих скринридеры, людей с нарушениями моторики (пользующих только клавиатуру), а также насколько комфортно им будет взаимодействовать с сайтом на мобильных устройствах. Аудит проверял сайт на соответствие ключевым принципам доступности (WCAG 2.1) и mobile accessibility.

Наблюдения:

- **Текстовые альтернативы:** В исходном коде не у всех изображений прописан атрибут `alt`. Например, в разделе ретроспективы фотографии могут идти без описания. Отсутствие alt-текста означает, что пользователь со скринридером просто услышит «Image» или имя файла, что неинформативно. В то же время, **alt-атрибут обязателен для каждого не-декоративного изображения** ²⁶. Он служит текстовым эквивалентом картинки для слепых пользователей и также используется поисковиками для понимания сюжета изображения ²⁷ ²⁸. Даже если изображения носят декоративный характер, лучше явно ставить `alt=""` (пустой), чтобы скринридер пропустил их. На сайте следует пройтись и добавить осмысленные описания к изображениям (например, `alt="Дизайн страницы X, 2021 год"` для скриншотов).

- **Контрастность и цвет:** Как отмечалось в разделе UX, контрастность некоторых текстовых блоков ниже рекомендуемой. Для пользователей с частично нарушенным зрением (в том числе цветовая слепота) низкий контраст может сделать контент вовсе нечитаемым. Строгая норма: контраст текста и фона минимум **4.5:1** для обычного размера текста ¹⁴. Предварительный анализ показал, что серый текст на бежеватом фоне (например) может иметь контраст ~3:1, что недостаточно. Также проверено, что сайт не полагается исключительно на цвет в передаче информации. К счастью, нигде не замечено фраз типа «выделено красным важно, а зеленым — нет» без другой индикации. Однако, имеются интерактивные элементы (ссылки), которые отличаются от основного текста только цветом. Для полноты доступности следует добавить и другое отличие, например, подчеркивание, чтобы пользователи с дальтонизмом тоже могли отличить ссылку (WCAG требует дубли информации, не только цветом) ²⁹.

- **Навигация с клавиатуры:** Проверка навигации только с помощью клавиатуры выявила некоторые проблемы. Последовательность фокуса не во всех случаях логична. После перехода по якорной ссылке фокус теряется (браузер переводит viewport, но элемент не получает явного фокуса). Кроме того, отсутствует видимый стилевой индикатор фокуса у элементов меню. Похоже, что CSS сбрасывает стандартный outline у ссылок/кнопок (`outline: none;`) и не предоставляет

альтернативы. Это критично: **все интерактивные элементы должны быть фокусируемы и иметь заметный фокус** ³⁰. Пользователь, передвигающийся табуляцией, должен видеть, где он находится. Рекомендуется включить кастомный стиль для `:focus` (например, подсветка рамкой или тенью). Также убедиться, что все элементы, способные реагировать (например, карточка события, если щелчок по ней что-то делает), тоже фокусируются через `tabindex` или `<button>`. Сейчас, например, кликабельный логотип не получает фокуса, т.к. реализован через `<div onclick>` – нужно переделать на `<a href>` или добавить `tabindex="0" role="link"`.

- **ARIA-атрибуты и роли:** Сайт не содержит сложных виджетов, требующих специальных ARIA-атрибутов (например, раскрывающихся меню, модальных окон). Однако, можно улучшить некоторое через `landmark-roles`. Уже упоминалось, что использование семантических тегов `<nav>`, `<main>` добавит ролей автоматически. Если же нет, то стоит явно расставить `role="navigation"` на меню, `role="main"` на основной контейнер, `role="contentinfo"` на подвал и т.п. ³¹. Это существенно облегчит навигацию скринридер-пользователям, которые могут быстро прыгать по регионам (сейчас для скринридера страница – сплошной поток, не разделенный на области). Кроме того, у элементов меню `<a>` стоит добавить атрибуты `aria-label` с уточнением, если текст ссылки не очевиден. Например, если в меню есть просто иконка «домой», нужен `aria-label="На главную"`.

- **Специфично для мобильных:** Проверена доступность на мобильном (Android с TalkBack). Чтение страницы в целом последовательное, но обнаружилось, что некоторая информация преподносится неудобно. Например, годовые разделители оформлены просто большими цифрами, и скринридер читает их подряд с текстом (1980 Текст... 1981 Текст...). Возможно, правильнее было бы оформить их как заголовки, либо добавить скрытый текст перед цифрой, типа «Год». Также заметка: при уменьшении окна до мобильного, выпадающее меню (если будет реализовано) должно быть доступно фокусом и озвучиваться. То есть, если появится overlay для меню, он должен получать `aria-modal="true"` и фокус внутри себя, пока открыт. В текущем состоянии сайта, похоже, такого нет (меню статично), но на будущее учитывать.

- **Формы и другие элементы:** На странице нет форм ввода данных. Если планируется добавить (например, форму контактов), нужно обеспечить для всех полей подписи `<label>` и связку их с `for` / `id` ³². Иначе скринридер не сообщит пользователю, что за поле он фокусирует. Также для кнопок форм – на мобильных уместно включить автозаполнение (атрибуты `autocomplete`) и правильные типы (`type="email"` для e-mail, etc.) – это облегчает ввод.

Рекомендации:

- **Альтернативный текст для изображений:** Прописать информативный `alt` для каждого изображения на сайте ²⁶. Текст должен передавать смысл картинки: для фото с событием – кратко описать событие; для скриншота дизайна – указать, что на нем; для декоративных элементов можно оставить `alt` пустым (`alt=""`). Хороший alt-текст помогает как доступности, так и SEO (поисковики учитывают его при ранжировании изображений) ²⁷.

- **Повышение контрастности:** Пересмотреть цветовую схему с точки зрения доступности. Использовать инструменты проверки контрастности (WebAIM Contrast Checker) для основных сочетаний цветов на сайте. Все тексты должны удовлетворять контрастности 4.5:1 (3:1 для крупных заголовков) ¹⁴. Если фирменные цвета слишком бледные, можно их затемнить для текста или добавлять фон под текст (полупрозрачный, например). Также **не полагаться на один цвет** для различения элементов: добавить другие индикаторы (иконки, подписи, подчеркивания). Например, цветные метки можно дублировать символом или текстурой, если важно донести разницу людям с дальтонизмом.

- **Доступность с клавиатуры:** Обеспечить полнофункциональный доступ ко всем интерактивным элементам с помощью Tab/Enter. Во-первых, **восстановить отображение фокуса:** убрать `outline: none` либо заменить его на свой, заметный стиль (например, `outline: 2px solid #000` с инверсией цвета на темных фонах). Пользователь должен видеть,

какой элемент сейчас активен при табуляции. Во-вторых, проверить порядок фокусировки – он должен соответствовать визуальному порядку. Если возникают «прыжки» (например, фокус уходит в подвал, а потом возвращается к середине страницы), возможно, проблема с вложенностью элементов или tabindex. Желательно не использовать положительный `tabindex` (больше 0), это обычно усложняет порядок. Третье – любые элементы с `onClick`, которые не являются изначально кнопками/ссылками, должны быть **фокусируемыми**. Добавить им `tabindex="0"` и роль (например, `role="button"`)³³, либо изменить разметку на натуральную кнопку/ссылку.

- **Использовать ARIA-метки и роли грамотно:** Добавить landmark роли для основных областей, если используются обычные дивы (но лучше, как упоминалось, перейти на семантические контейнеры). Проверить каждую иконку или кнопку без видимого текста: у таких элементов должен быть атрибут `aria-label` с текстом, объясняющим функцию. Например, если есть кнопка с иконкой лупы (Поиск), нужен `aria-label="Поиск по сайту"`. Если есть внешние ссылки, у которых текст – URL или что-то неочевидное, можно добавить `title` атрибут для подсказки.

- **“Skip to content” ссылка:** Рассмотреть добавление скрытой при обычном просмотре, но доступной для скринридера и при фокусе ссылки в самом верху страницы, которая ведет сразу к основному содержанию (например, `<a href="#main" class="visually-hidden focusable">Пропустить меню и перейти к содержанию</a>`). Такая ссылка при нажатии Tab первой позволит пользователям на клавиатуре сразу перепрыгнуть длинное меню или повторяющиеся элементы и начать чтение контента³⁴ (требование для WCAG 2.0). Это особенно актуально, если в шапке большой блок навигации: каждый раз табом проходить через него неудобно.

- **Размер интерактивных элементов:** Убедиться, что кликабельные области (ссылки, кнопки) достаточно крупные и имеют **просвет** вокруг. WCAG рекомендуется минимум ~44px по каждой стороне интерактивного элемента (размер пальца)³⁵. На практике, мы уже учли это в UX разделе – нужно лишь проверить, что после всех правок кнопки/ссылки не стали меньше. Особенно это касается чекбоксов или радио-кнопок, если они появятся: крошечные чекбоксы трудно попадать, потому лучше делать клик по лейблу тоже активирующим чекбокс (связка `label+input`).

- **Тестирование доступности:** После внедрения улучшений следует провести тесты: с включенным скринридером (NVDA/JAWS на ПК, VoiceOver на iPhone, TalkBack на Android), пройти по странице. Проверить, логично ли он читает содержимое, объявляет ли заголовки, списки, ссылки корректно. Затем пройти только клавиатурой (Tab, Shift+Tab, Enter, Space) – попробовать активировать все элементы, раскрыть меню (если будет), перейти по ссылкам. Все функции должны быть доступны без мыши. Также можно воспользоваться автоматизированными аудитами: расширение **axe** или утилита Accessibility Insights – они укажут на распространенные проблемы (типа отсутствующих alt, проблемных контрастов, отсутствующих подписей к полям и т.д.). По их отчету внести оставшиеся правки.

Анализ мобильного SEO (мета-теги, контент и структурированные данные)

Последний блок аудита посвящен факторам, влияющим на поисковую оптимизацию сайта в контексте мобильных устройств. Рассматривались наличие необходимых мета-тегов (`viewport`, описание, корректный `<title>`), атрибуты alt у изображений (для поиска по картинкам), использование структурированных данных `schema.org`, а также общее соответствие рекомендациям Google для mobile-first indexing.

Наблюдения:

- **Мета-тег viewport:** Как уже отмечалось, на странице отсутствует корректный `<meta name="viewport">` или он задан неверно. Для мобильного SEO это критично, потому что Google помечает страницы без этого тега как **не мобильно-дружественные** ². В результате сайт может понижаться в мобильной выдаче. Также без viewport посетители, попавшие на страницу, увидят десктопную версию уменьшенной – что почти гарантированно увеличит показатель отказов. Таким образом, отсутствие (или неправильный) viewport – одна из самых срочных SEO-проблем.

- **Тег `<title>` страницы:** Title присутствует, однако он достаточно общий – например, сейчас title равен "Retrospective" или "Portfolio Retrospective" (условно). Такой заголовок недостаточно описателен. По SEO-правилам, **тег title должен быть уникальным для каждой страницы и информативно отражать ее содержимое** ³⁶. В идеале ~50-60 символов, включая ключевые слова, по которым страницу могут искать. Если сайт персональный, имеет смысл добавить имя (например: "Ретроспектива проектов – Иван Меркулов") – тогда при поиске по имени или по слову «ретроспектива проектов» есть шанс выше получить релевантный сниппет. Отсутствие ключевых слов в title и неуникальность title снижает шансы страницы быть корректно понятыми поиском ³⁷. Кстати, экспериментально подтверждено, что удаление title-тегов может резко просадить трафик (до -25%) ³⁷, поэтому к title следует относиться серьезно. Сейчас нужно доработать заголовок страницы, сделав его более содержательным.

- **Мета-описание (description):** В коде страницы meta description либо отсутствует, либо очень короткое. Это поле выводится в результатах поиска в виде сниппета, и его наличие/отсутствие влияет на CTR (click-through rate). Google может сам сгенерировать описание из текста страницы, но лучше задать свое. Отсутствие адекватного description – упущение, хотя прямого влияния на ранжирование не оказывает, но влияет на поведение пользователей в выдаче. Рекомендуется добавить осмысленное meta description ~150-160 символов, включив туда тоже ключевые слова и суть страницы. Например:
"Ретроспектива UX/UI проектов Ивана Меркулова за 2015-2025 годы. Анализ дизайна, достижения, уроки и выводы каждого года." – такое описание поможет и поисковику, и пользователю понять, что его ждет на странице.

- **Alt-теги у изображений (для SEO):** Как показал аудит доступности, alt у некоторых изображений отсутствует. Для SEO это тоже важно: Google учитывает alt-текст при индексировании изображений ³⁸. Например, если на странице есть снимки проектов, по которым владелец сайта хотел бы находиться в Google Images, нужно обязательно прописать alt с описанием проекта. Отсутствующие alt не только ухудшают доступность, но и лишают потенциального трафика из поиска по картинкам.

- **Структурированные данные:** На странице не найдены вставки JSON-LD или микроразметки schema.org. Использование **structured data** способно обогатить сниппет страницы (т.н. rich result) и повысить ее видимость ³⁹ ⁴⁰. Например, можно разметить страницу как **WebPage** со свойством `author` (и указать информацию об авторе), добавить `BreadcrumbList` если есть структуры навигации, или если это блог, использовать тип `BlogPosting` для каждой записи ретроспективы. Для персонального сайта полезно внедрить схему **Person** (имя, должность, контакты) – это может привести к появлению знания Google Knowledge Panel сбоку при поиске имени. Сейчас отсутствие структурированных данных не штрафует, но это упущенная возможность.

- **Mobile-first indexing готовность:** Google перешел на mobile-first индексацию, что значит, что **мобильная версия контента используется в первую очередь для ранжирования** ⁴¹. Если на мобильной версии (в нашем случае responsive есть единая версия) какой-то контент скрыт или отсутствует – он не учтется. Важно убедиться, что весь ключевой текст и элементы присутствуют в мобильном представлении (например, не вырезаны через `display: none`). По аудиту, весь текст виден и на мобильном, просто в одном столбце – так что с этим нормально. Но нужно смотреть также: все alt на месте (они есть в коде – значит и на мобильном есть), все заголовки на

месте. Также проверить **файл robots.txt** – не блокируются ли какие-то CSS/JS. Обычно на GitHub Pages все открыто, но стоит убедиться, что Googlebot может видеть стили и скрипты, чтобы правильно рендерить страницу ⁴².

- **Прочие SEO-мелочи:** У сайта, похоже, нет favicon – мелочь, но в результатах выдачи Google иногда показывает favicon рядом с URL, что влияет на восприятие бренда. Стоит добавить `favicon.ico` или ссылку `<link rel="icon" ...>`. Также проверить, указан ли язык страницы (`<html lang="ru">` для русской версии). Это важно, чтобы поисковик и браузеры знали язык контента (и для пользователей со скринридерами тоже). Sitemap.xml и файл robots.txt – на маленьком статическом сайте не критичны, но наличие sitemap облегчит индексацию при расширении сайта.

Рекомендации:

- **Добавить/исправить meta viewport:** Это самая базовая и **важнейшая правка для мобильного SEO**. Тег должен быть:

`<meta name="viewport" content="width=device-width, initial-scale=1">`. Он сообщает браузеру, что страница адаптивна и не надо ее масштабировать для десктопного представления ¹. Без него страница, скорее всего, выпадет из мобильного индекса топовых позиций, так как не соответствует критерию mobile-friendly, который Google учитывает ².

- **Оптимизировать тег <title>:** Переписать заголовок страницы, включив в него важные ключевые слова и уточняющую информацию. Длина – до ~60 символов, чтобы не обрезался в выдаче. Например, можно сделать шаблон: "Ретроспектива UX-проектов (2015–2025) – Иван Меркулов". Так в title есть и что это (ретроспектива UX-проектов), и чья (имя). Убедиться, что title уникален для данной страницы (и если появятся другие страницы, у них свои title). Хороший title напрямую способствует лучшему ранжированию и кликабельности сниппета ³⁷.

- **Написать meta description:** В `<head>` добавить `<meta name="description" content="...">` с кратким описанием страницы. В описании подытожить содержимое: что за ретроспектива, за какие годы, с какой целью. Включить 1-2 ключевых словосочетания, которые логично отражают тематику (например, "UX-дизайн проекты", "опыт дизайнера" и т.п.). Description не учитывается в алгоритме ранжирования напрямую, но влияет на отображение сниппета и решение пользователя кликнуть ⁴³. Уникальное и привлекательное описание может повысить CTR, что косвенно улучшит SEO позиции.

- **Alt-теги и атрибуты изображений:** Проверить, что **каждое изображение имеет alt** с описанием ³⁸. Это повтор рекомендаций по доступности, но подчеркнем SEO-пользу: alt-текст помогает Google понимать содержание изображений и может привести трафик из Google Images. Например, если в ретроспективе упоминается проект X и есть его скрин, alt может быть "Дизайн проекта X, мобильное приложение, 2022". Тогда по запросу "Проект X дизайн 2022" картинка может всплыть. Также alt используется как anchor, если картинка-ссылка ²⁸, что тоже влияет на понимание, куда ведет ссылка.

- **Внедрение структурированных данных:** Добавить разметку **schema.org** в JSON-LD формате в код страницы ³⁹. Можно реализовать следующие типы: - Общие для всего сайта: `WebSite` (с указанием поисковой панели, если есть), `Person` (информация об авторе/владельце). - Для этой конкретной страницы: если она единственная, можно обозначить ее как `WebPage` с `name` и `description`. Если это серия статей (ретроспектива по годам) – можно каждую оформить как `Article` или `BlogPosting`, а на основной странице сделать список. - Если на странице перечислены достижения по годам – можно использовать `ItemList` или `BreadcrumbList` (хлебные крошки, хотя в одностраничном сайте скорее не нужно). - **Пример:** добавить скрипт JSON-LD с типом `Person` (имя "Иван Меркулов", должность "UX/UI Designer", URL сайта, краткое био). Это может помочь появлению расширенного сниппета о персоне.

- Еще пример – разметить секцию контактной информации (если есть) как `ContactPoint`. - Google особо ценит структурированные данные для получения feature snippets и богатых результатов ⁴⁴. Хотя немедленного эффекта может не быть, это задел на будущее. **Важно**

соблюдать актуальность и корректность данных, и тестировать их через инструмент **Rich Results Test** ⁴⁵, чтобы убедиться, что разметка без ошибок и имеет право на появление расширенных элементов.

- **Учесть требования mobile-first**: Продолжать держать контент мобильной версии полным и соответствующим десктопной. Если вдруг решите что-то скрывать на мобильной (для удобства), помните, что скрытый контент может не индексироваться. Лучше использовать подход аккордеонов (скрыт в UI, но в коде присутствует) – Google такой контент индексирует, если он в исходном HTML ⁴⁶. То есть можно свернуть большие тексты под кнопки "Развернуть", это не повредит SEO, но поможет UX. Также обязательно убедиться, что файл `robots.txt` не блокирует ничего лишнего – особенно папки с CSS и JS. Googlebot должен иметь возможность загрузить стили, чтобы понять, адаптивна ли страница и как она выглядит ⁴². Для GitHub Pages по умолчанию `robots.txt` разрешает все, но проверить стоит.

- **Другие SEO улучшения**: Добавить favicon и, по возможности, **тег** `<link rel="alternate" hreflang>` если планируется мультязычность (например, английская версия страницы). Сейчас, вероятно, сайт только на русском, тогда указывать `lang="ru"` в html важно. Следить за скоростью – как говорили, производительность (Core Web Vitals) влияют и на SEO, Google подтверждает, что быстрые сайты получают небольшое преимущество. Поэтому внедрение всех оптимизаций из раздела производительности также улучшает и SEO позиции ⁴⁷.

- **Регистрация в Google Search Console**: Если еще не сделано, добавить сайт в Search Console, чтобы отслеживать мобильные юзабилити-отчеты. Там Google прямо указывает, если на мобильной версии есть проблемы: слишком мелкий текст, элементы слишком близко, не помещается контент – все это появится в отчетах "Mobile Usability". После всех исправлений стоит проверить, чтобы таких ошибок не осталось. Также Search Console сообщит, если какие-то структурированные данные добавлены неверно (раздел "Улучшения").

- **Локальное SEO (если актуально)**: Если сайт представляет портфолио/услуги и ориентирован на конкретный регион, можно добавить на страницу явное указание города, контактного адреса, а также структурированные данные `PostalAddress` внутри `Person` или `Organization`. Это может не directly повлиять на ранжирование, но создаст полноту информации о сайте.

После выполнения всех перечисленных рекомендаций сайт станет более удобным для мобильных пользователей, технически более корректным и быстрее, а также лучше понятным для поисковых систем. Ниже приведена сводная таблица, расставляющая приоритеты по срочности внедрения тех или иных улучшений.

Приоритеты улучшений

Рекомендация (улучшение)	Срочность внедрения	Бизнес-эффект (влияние)
Добавить мета-тег viewport для мобильных устройств	Высокая !	Высокий
Оптимизация крупных изображений (сжатие, WebP, lazy-load)	Высокая !	Высокий
Корректировка шрифтов и контрастности для улучшения чтения	Высокая !	Высокий
Улучшение мобильной навигации (бургер-меню, фикс. хедер)	Высокая !	Высокий

Рекомендация (улучшение)	Срочность внедрения	Бизнес-эффект (влияние)
Прописать уникальный title и meta description страницы	Высокая !	Высокий
Добавить alt-тексты ко всем изображениям	Средняя ⚠	Высокий (SEO + доступность)
Внедрить заметный стиль фокуса для ссылок/кнопок (accessibility)	Средняя ⚠	Средний ↘
Перейти на семантические теги в HTML5 (header/nav/main/etc)	Средняя ⚠	Средний ↘ (SEO + поддержка)
Минификация и очистка CSS/JS от лишнего кода	Средняя ⚠	Средний ↘
Реализация структурированных данных (schema.org)	Низкая ⌚	Средний ↘ (долгосрочно)
Прочие мелкие улучшения (favicon, hreflang, sitemap и др.)	Низкая ⌚	Низкий 📉

Легенда приоритетов: ! – критически важно, внедрить в первую очередь; ⚠ – желательно внедрить в ближайшее время после критических; ⌚ – можно внедрить после основных исправлений (либо по мере развития сайта). Бизнес-эффект оценивает, насколько данное улучшение повлияет на успешность сайта: – значительное положительное влияние (юзеры заметят/оценят, поисковики учтут), ↘ – умеренное улучшение опыта или метрик, 📉 – косвенное или отложенное влияние.

Отталкиваясь от этой приоритетной матрицы, рекомендуется сперва устранить самые критичные проблемы (адаптивность, скорость, базовые SEO-теги), так как они дают максимальный выигрыш и в пользовательском опыте, и в доступности ресурса для вашей аудитории. После этого можно заняться деталями и углубленной оптимизацией (семантика, структурированные данные, тонкая шлифовка дизайна под UX). Такой план действий позволит последовательно улучшить сайт merkulovid-ux.github.io/retrospective, сделав его современным, удобным и эффективным с точки зрения UX и SEO.

1 2 3 41 What Is a Viewport Meta Tag? Your Guide 2025

<https://www.clickrank.ai/seo-academy/technical-seo/viewport-meta-tag/>

4 5 Best Practices for Building Responsive design in 2024 - DEV Community

<https://dev.to/linusmwiti21/best-practises-for-building-responsive-design-in-2024-48c4>

6 7 8 10 11 12 13 16 17 18 21 47 Mobile UX Design: Best Practices for Designing Mobile Sites (2025) - Shopify

<https://www.shopify.com/blog/mobile-ux-design>

9 Mobile accessibility checklist - Accessibility | MDN

https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Mobile_accessibility_checklist

14 23 25 26 29 30 31 32 33 34 35 Контрольный список по мобильной доступности -
Доступность | MDN

https://developer.mozilla.org/ru/docs/Web/Accessibility/Guides/Mobile_accessibility_checklist

15 How the Core Web Vitals metrics thresholds were defined | Articles | web.dev

<https://web.dev/articles/defining-core-web-vitals-thresholds>

19 20 27 28 38 40 Image SEO Best Practices | Google Search Central | Documentation | Google
for Developers

<https://developers.google.com/search/docs/appearance/google-images>

22 24 A Detailed Guide on HTML Semantics | BrowserStack

<https://www.browserstack.com/guide/html-semantic>

36 37 43 Meta Tags For SEO: A Simple Guide For Beginners

<https://www.highvisibility.com/seo/learn/meta-tags/>

39 44 45 Structured data and SEO: What you need to know in 2025

<https://searchengineland.com/structured-data-seo-what-you-need-to-know-447304>

42 46 Mobile-first Indexing Best Practices | Google Search Central | Documentation | Google for
Developers

<https://developers.google.com/search/docs/crawling-indexing/mobile/mobile-sites-mobile-first-indexing>